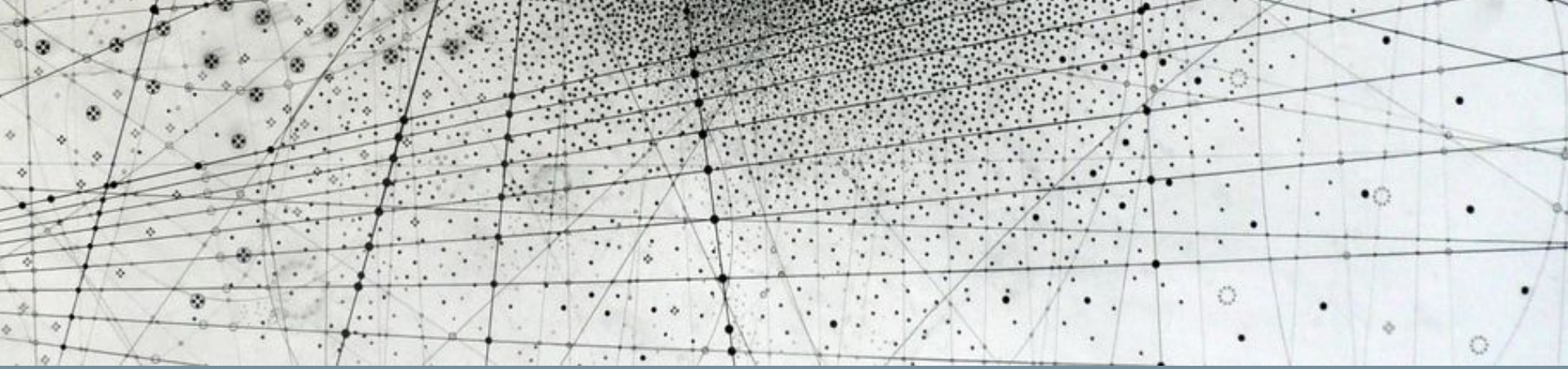
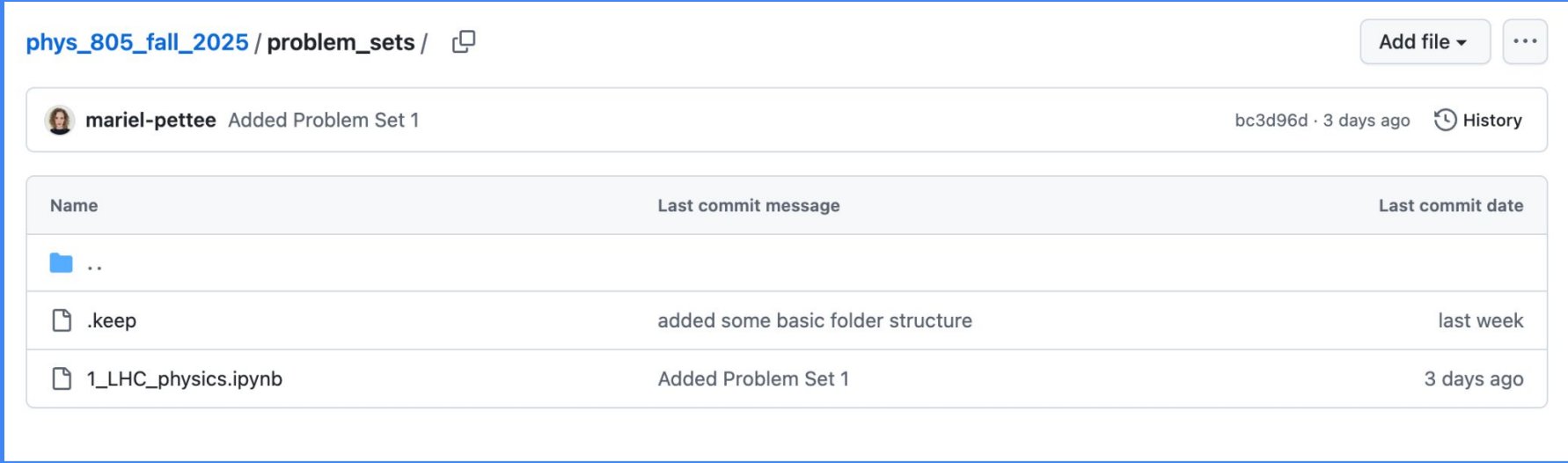


Research Methods in Machine Learning & Physics

Problem set 1 has been posted – due Friday end of day:
(Make sure to not change the random seed values!)

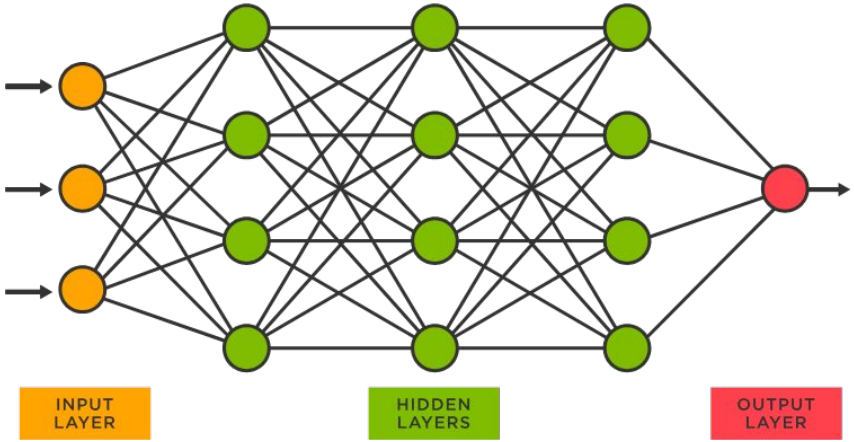


Then, Project 1 will be posted on Friday and due after 2 weeks.

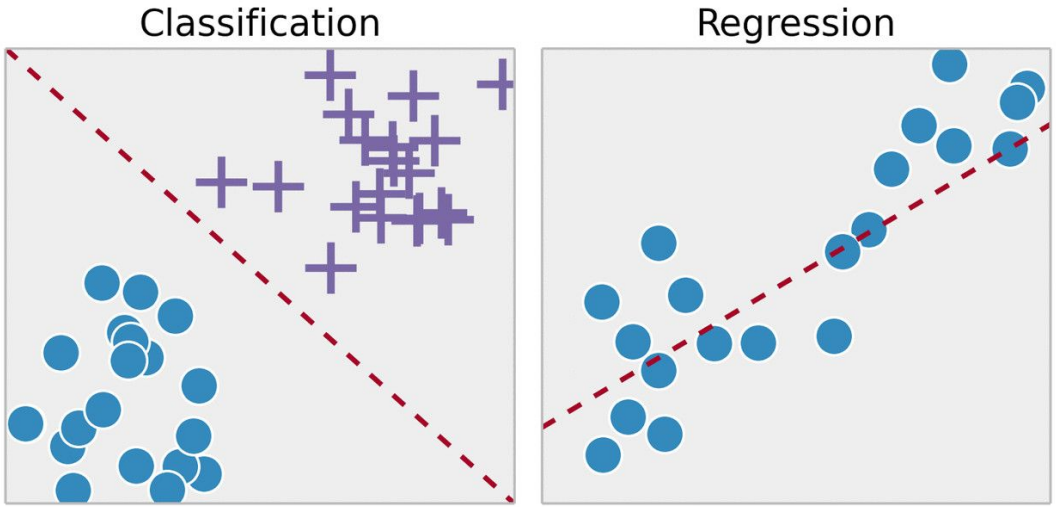
What can you do with a basic neural network?

Dense neural networks (DNNs)

- **Also called:**
  - Fully-connected neural networks (FCNNs)
  - “Vanilla” neural networks
  - Multi-layer perceptrons (MLPs)



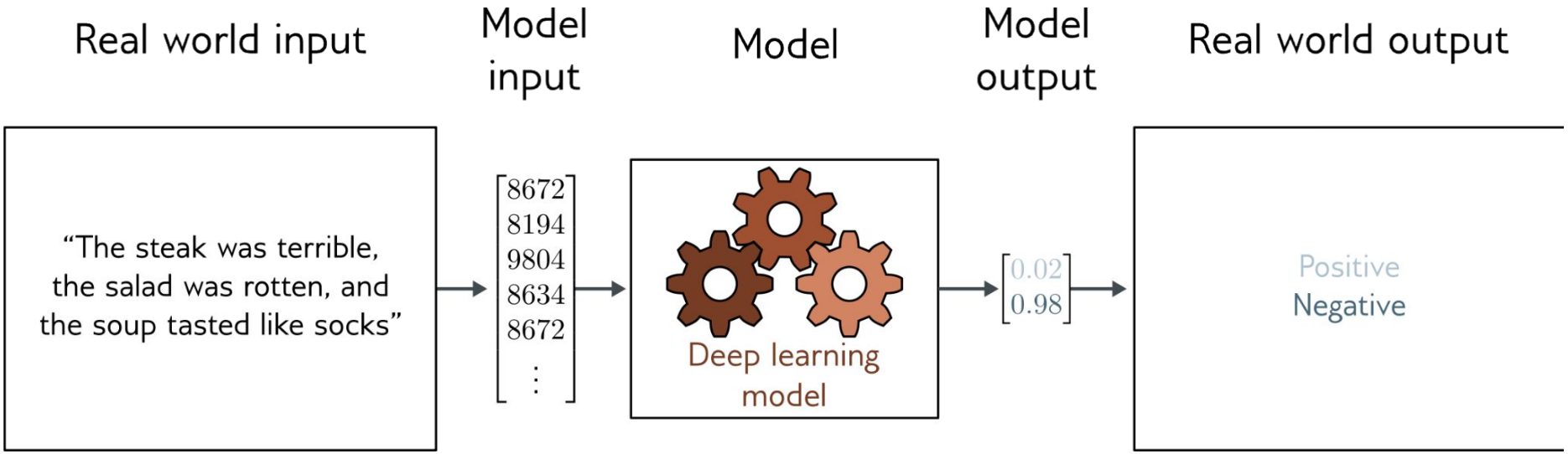
Types of tasks a NN can solve



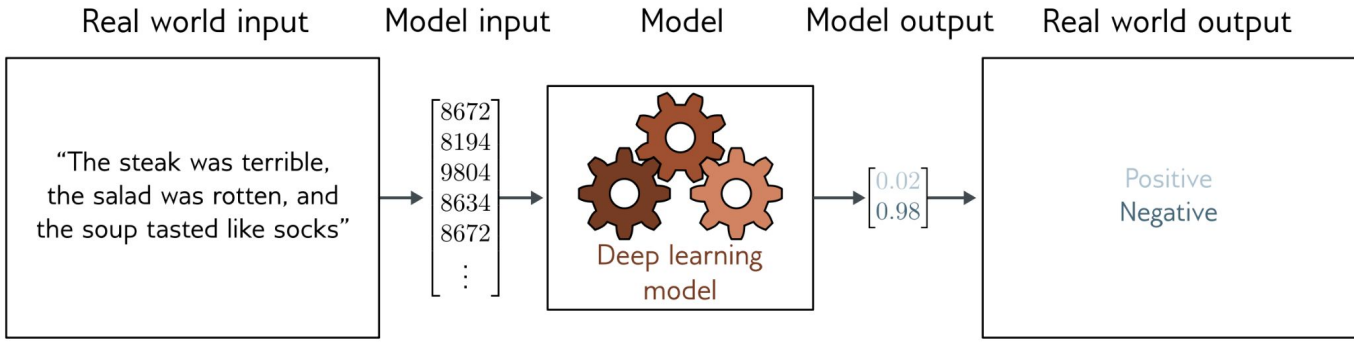
Predictions are probability distributions over discrete class labels

Predictions are continuous real-valued numbers

Classification



Common classification losses



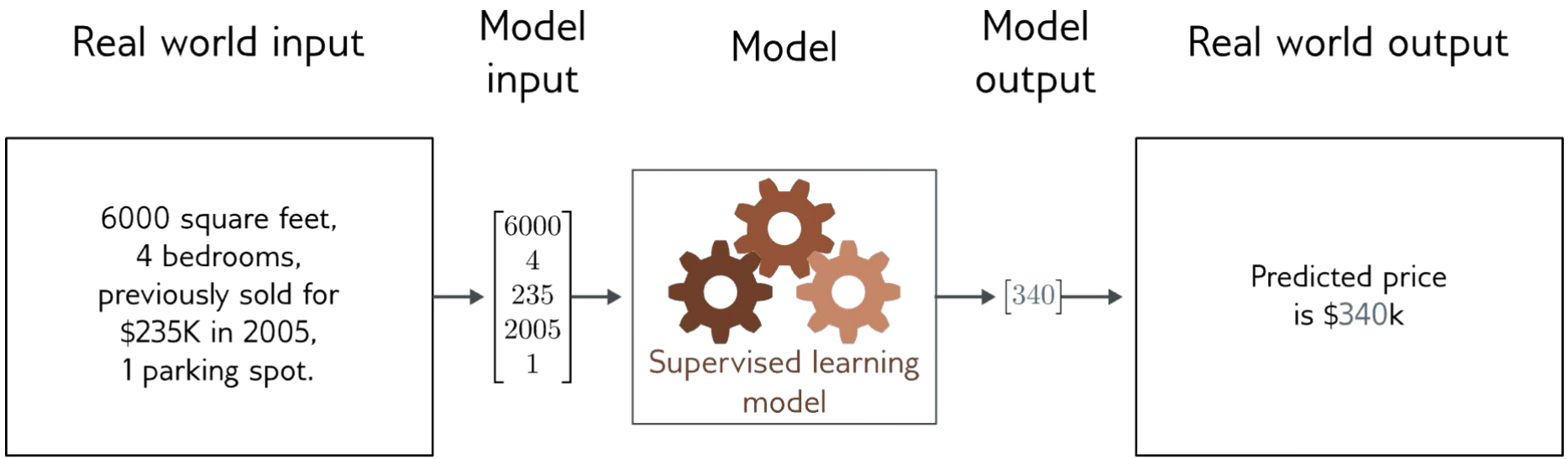
Binary cross-entropy

$$l_n = -w_n \left[y_n \cdot \log x_n + (1 - y_n) \cdot \log(1 - x_n) \right]$$

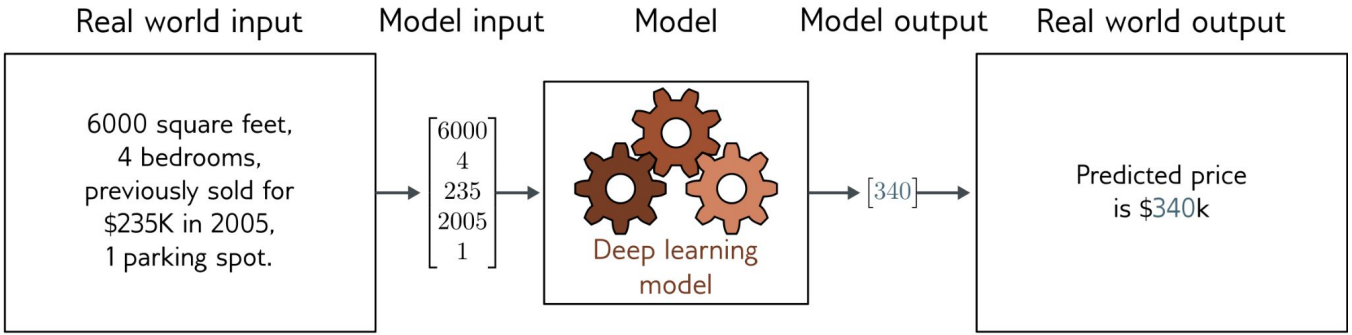
Categorical cross-entropy

$$l_n = -w_{y_n} \log \frac{\exp(x_{n,y_n})}{\sum_{c=1}^{C} \exp(x_{n,c})}$$

Regression


Real world input
Model input
Model
Model output
Real world output
6000 square feet,
4 bedrooms,
previously sold for
$235K in 2005,
1 parking spot.
6000
4
235
2005
1
Supervised learning model
340
Predicted price
is $340k

Common regression losses



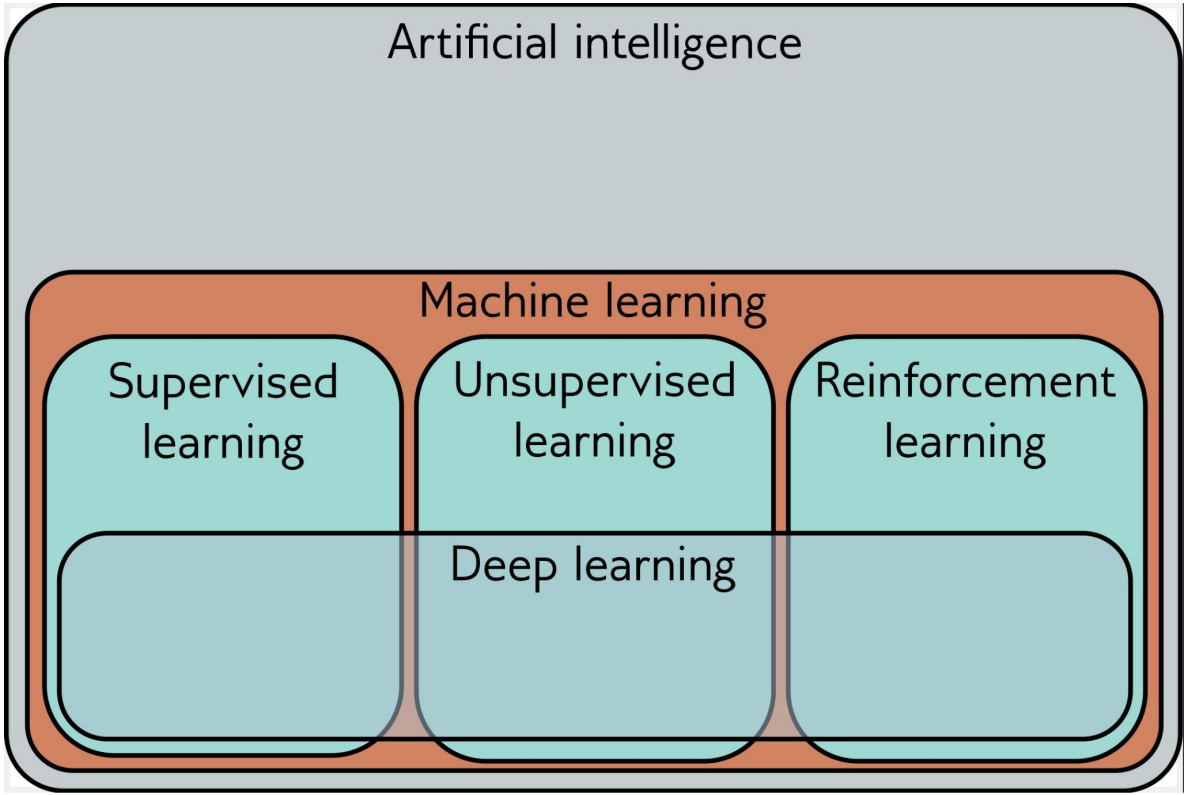
Mean squared error (MSE)
(a.k.a. L2Loss)

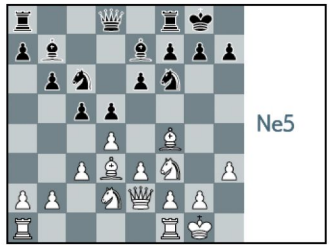
$$l_n = (x_n - y_n)^2$$

Mean absolute error (MAE)
(a.k.a. L1Loss)

$$l_n = |x_n - y_n|$$

Types of ML


Artificial intelligence
Machine learning
Supervised learning
Unsupervised learning
Reinforcement learning
Deep learning

Supervised learning	Unsupervised learning	Reinforcement learning
		
Bicycle		Reward = 0
		
Apple		Reward = -1
		
Aardvark		Reward = +1

Other categories:

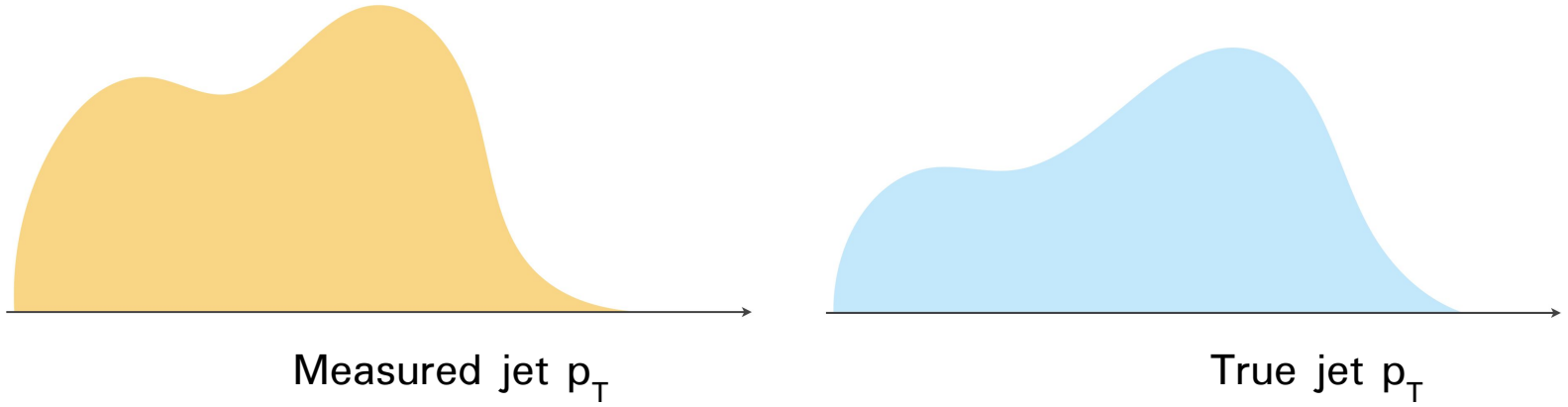
- Generative ML
  - Goal is to sample from a distribution to generate realistic novel examples of your training data

- Self-supervised ML
  - “Pretext” task that doesn’t precisely align with your end goal

- Weakly-supervised ML
  - Like supervised ML, but with missing or imperfect labels

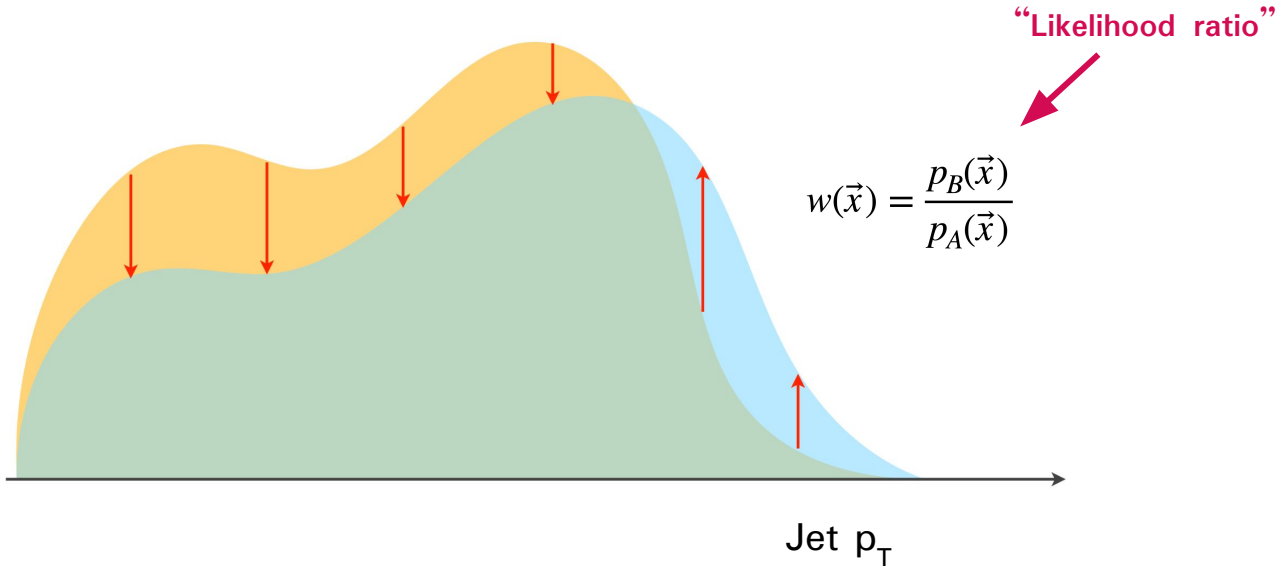
More subtle uses of classifiers

High-dimensional & unbinned unfolding

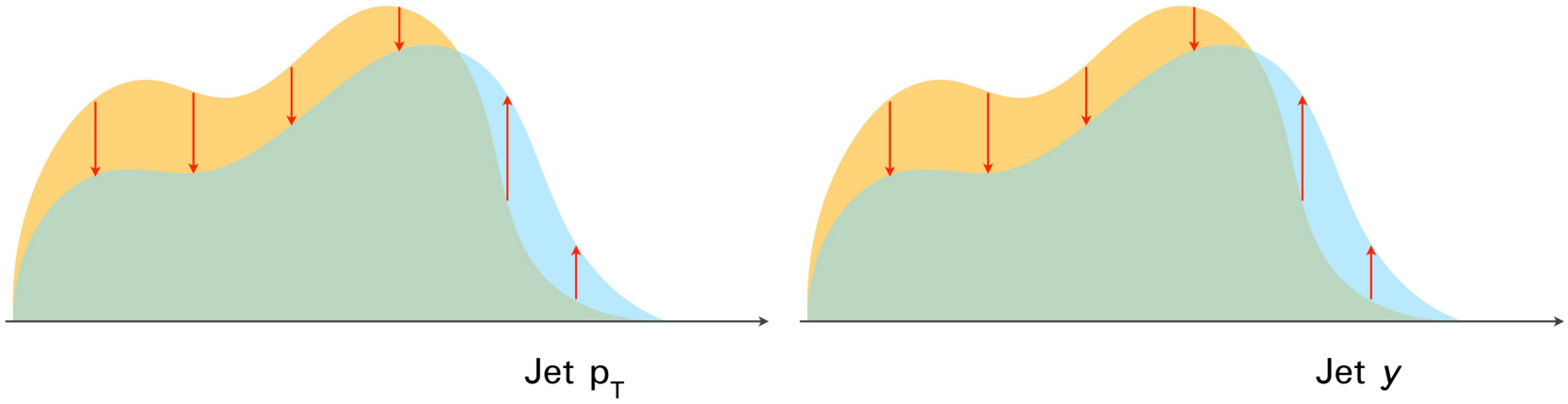


High-dimensional & unbinned unfolding

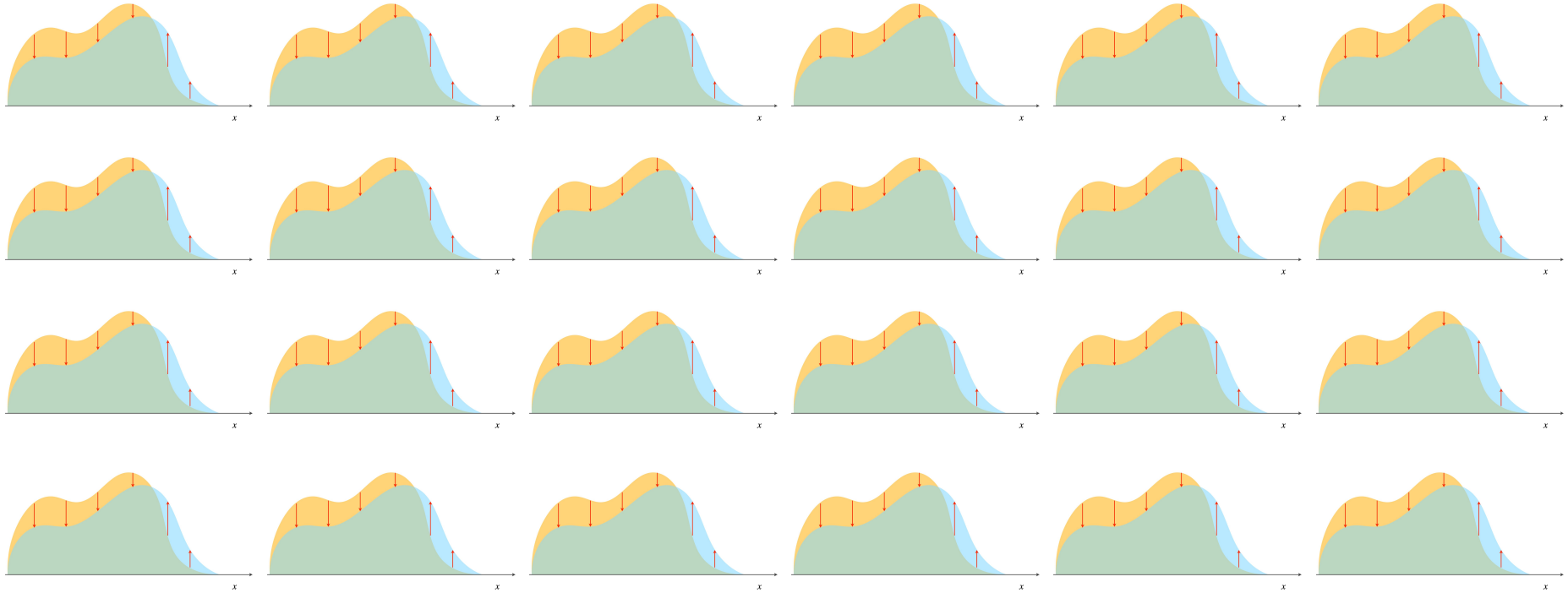
Learn a reweighting function based on the ratio of the probability densities.



High-dimensional & unbinned unfolding



High-dimensional & unbinned unfolding



Classifier functions can be re-used to directly approximate a likelihood ratio.

A vanilla NN classifying between two classes could be trained using **binary cross-entropy loss:**

$$\mathcal{L}_{\mathrm{BCE}}[f] = -\int d\vec{x}\,(p_A(\vec{x})\log(f(\vec{x})) + p_B(\vec{x})\log(1-f(\vec{x})))$$

The function that minimizes this expression will satisfy, for any small variation $\delta f(\vec{x})$:

$$\delta\mathcal{L}_{\mathrm{BCE}} = \int d\vec{x}\left(-\frac{p_A(\vec{x})}{f(\vec{x})} + \frac{p_B(\vec{x})}{1-f(\vec{x})}\right)\delta f(\vec{x}) = 0,$$

$$-\frac{p_A(\vec{x})}{f(\vec{x})} + \frac{p_B(\vec{x})}{1-f(\vec{x})} = 0$$

Rescaling of classifier output $\quad \dfrac{f(\vec{x})}{1-f(\vec{x})} = \dfrac{p_A(\vec{x})}{p_B(\vec{x})}$ Likelihood ratio

The “likelihood ratio trick” is not limited to the binary cross-entropy loss function.

In fact, there is an entire class of loss functionals that have this property.

$$L[f] = -\int dx \left(p(x|\theta_A)\, A(f(x)) + p(x|\theta_B)\, B(f(x)) \right)$$

Probability density A

Rescaling function

Probability density B

Rescaling function

$$L[f] = -\int dx \left(p(x|\theta_A)\, A(f(x)) + p(x|\theta_B)\, B(f(x)) \right)$$

Probability density A

Rescaling function

Probability density B

Rescaling function

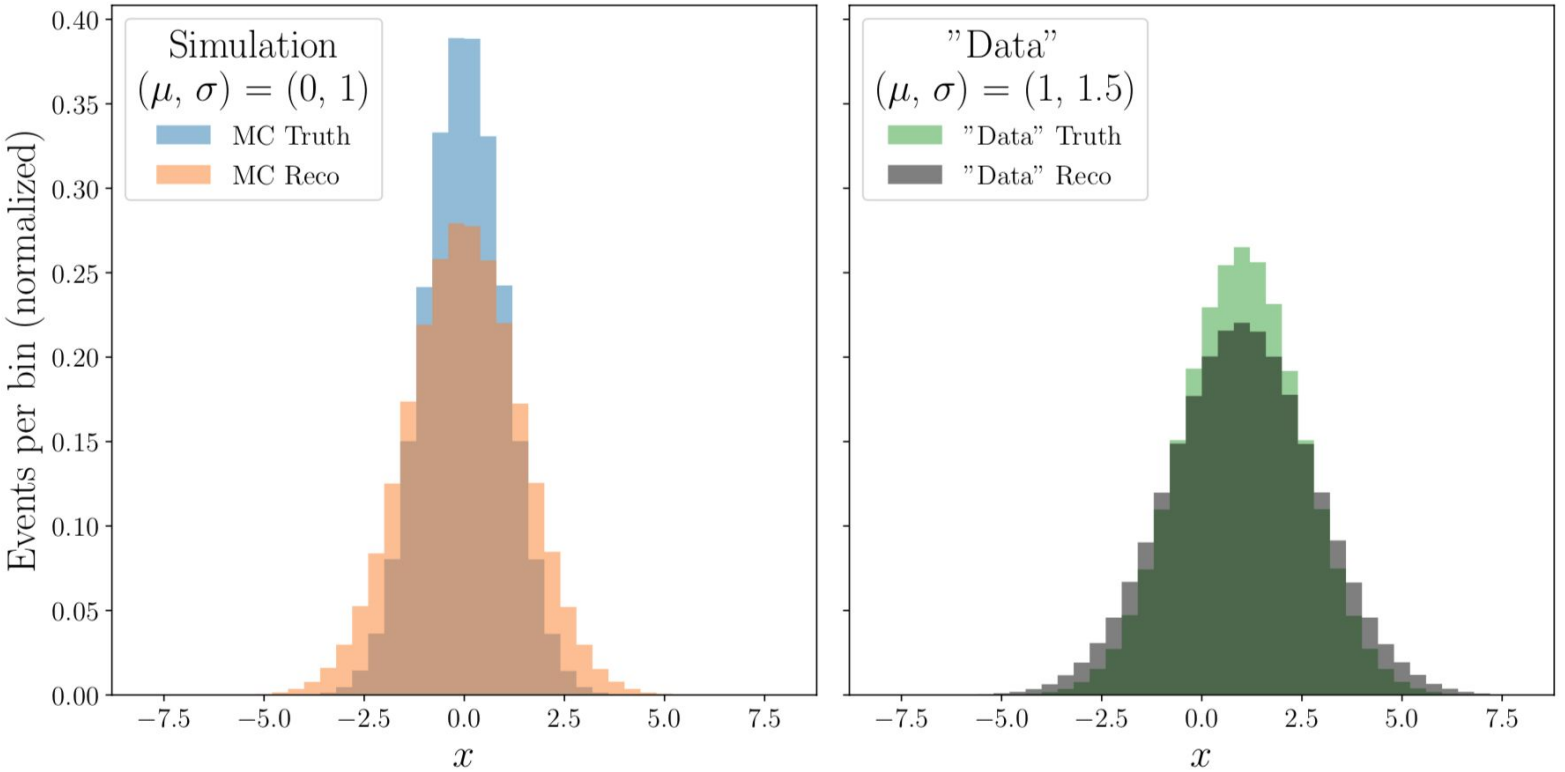
$$\frac{\delta L}{\delta f} = -\frac{\partial}{\partial f}\Big(p(x \mid \theta_0) A(f(x)) + p(x \mid \theta_1) B(f(x)) \Big) = 0 \iff -\frac{B'(f(x))}{A'(f(x))} = \frac{p(x \mid \theta_0)}{p(x \mid \theta_1)} = \mathcal{L}(x)$$

Must be a monotonic rescaling of *f*

$$L[f] = -\int dx \left(p(x|\theta_A)\, A(f(x)) + p(x|\theta_B)\; B(f(x)) \right)$$

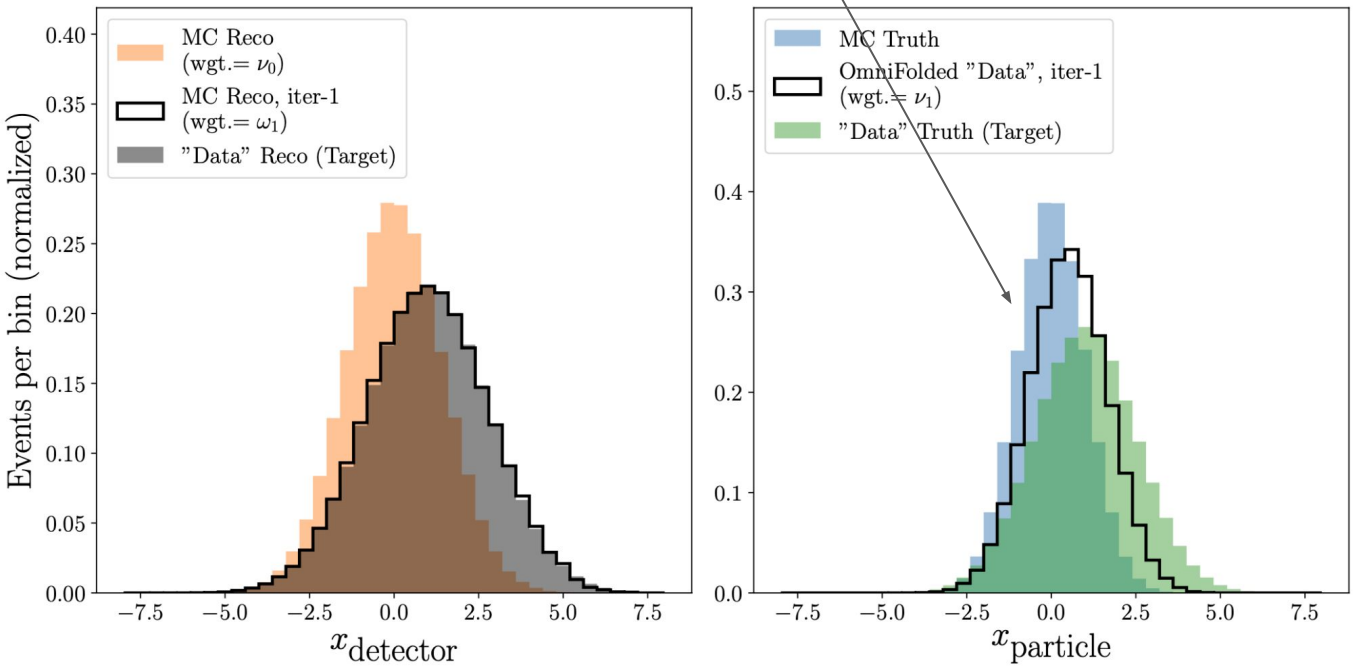
Loss Name	$A(f)$	$B(f)$
Binary Cross-Entropy	$\ln(f)$	$\ln(1-f)$
Mean Squared Error	$-(1-f)^2$	$-f^2$
Maximum Likelihood Classifier	$\ln(f)$	$1-f$
Square Root	$-\frac{1}{\sqrt{f}}$	$-\sqrt{f}$

Reweighting simulation to match data:



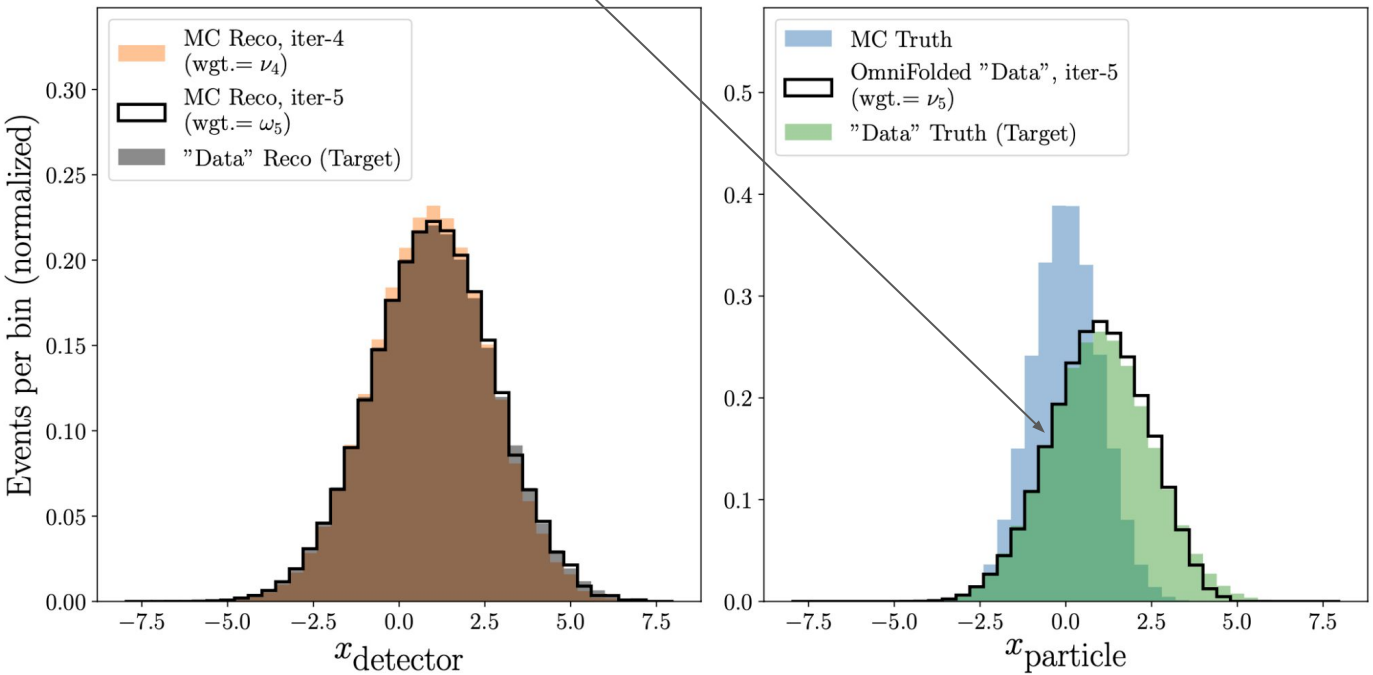
Reweighting simulation to match data:

After one iteration: truth-level simulation is **reweighted** to look more like truth-level data.



Reweighting simulation to match data:

After five iterations, the **reweighted** truth-level simulation is closely aligned with truth-level data.



Partner work: Training a Neural Network

- Navigate to the repository:
  - `https://github.com/mariel-pettee/phys_805_fall_2025`
- Work through Notebook 1
  - Don’t just read in silence! Have both partners look at one screen and discuss as you go.
- If you’re done early:
  - Please check out the notebooks in Chapter 5 of UDL here: `https://github.com/udlbook/udlbook/tree/main/Notebooks/Chap05`